SĂPTĂMÂNA 1

Introducere: sisteme, modele, simulare

Modelare și Simulare · Anul III, Calculatoare și Tehnologia Informației

Catalin Iapa · Anul universitar 2026–2027

Cursul pe scurt

10

săptămâni de curs

30

de ore: 3 ore pe săptămână

1

examen: grilă și răspunsuri scurte

- Laboratorul se desfășoară separat și aplică în Python ideile din curs
- Pentru fiecare săptămână: prezentare + suport de curs scris + întrebări de autoevaluare

Harta cursului

1 S1 Introducere

Sistem, model, simulare; etapele unui studiu

2 S2 Monte Carlo

Aleatorul în calculator; estimări prin eșantionare

3 S3 Markov

Plimbări aleatoare, lanțuri Markov, PageRank

4 S4 Cozi

Performanța serverelor și a sistemelor de calcul

5 S5 Evenimente discrete

Cum funcționează un simulator; SimPy

6 S6 Validare

Date de intrare, analiza rezultatelor, încredere

7 S7 Sisteme continue

Ecuații diferențiale, epidemii, integrare numerică

8 S8 Automate celulare

Modele spațiale, particule, calcul paralel

9 S9 Agenți

Comportament emergent; agenți cu LLM

10 S10 AI + simulare

Modele surogat, digital twins

Cum arată fiecare curs

1 Trei ore, trei teme

Fiecare oră are o temă proprie, cu obiective la început și rezumat la final.

2 Exemple concrete

Fiecare concept e legat de un sistem real: o bancă, un server, o epidemie, trafic.

3 Mini-demo în Python

Câteva linii de cod care arată ideea în acțiune – aceleași unelte ca la laborator.

4 Întrebări de verificare

Întrebări de tip grilă în timpul orei – exact stilul examenului.

Evaluare și resurse

Evaluare

- Examen final: întrebări grilă și răspunsuri scurte
- Corectare automată – contează înțelegerea conceptelor
- Întrebările de verificare din curs sunt model de examen
- Laboratorul are evaluarea lui separată

Resurse

- Prezentările și suportul de curs pentru fiecare săptămână
- A. M. Law – Simulation Modeling and Analysis
- A. B. Downey – Modeling and Simulation in Python (gratuită online)
- Python: NumPy, SciPy, SimPy, Mesa

Ora 1

De ce modelăm și simulăm?

1. Patru situații reale și ce au în comun
2. Sistem, entitate, atribut, stare, eveniment
3. Ce este un model și ce este o simulare
4. Când folosim simularea – și când nu

La finalul săptămânii veți putea…

1. să definiți **sistem**, **model** și **simulare** și să dați exemple
2. să descrieți un sistem prin **entități, atribute, stare și evenimente**
3. să **clasificați** un model: static/dinamic, determinist/stocastic, discret/continuu
4. să enumerați **etapele** unui studiu de simulare
5. să explicați diferența dintre **verificare** și **validare**
6. să dați exemple de **utilizări actuale** ale simulării

Magazinul online se așteaptă la de 10 ori mai mulți clienți de Black Friday. Câte servere închiriem?

— o întrebare de inginer, nu de matematician

Prea puține: site-ul cade. Prea multe: plătim degeaba. Nu putem „încerca” Black Friday de zece ori.

Patru situații reale

1 Trafic urban

Merită o bandă în plus sau un sens giratoriu? Construcția costă milioane și durează ani.

2 Epidemie

Cât de repede se răspândește un virus? Ce efect are închiderea școlilor? Nu putem experimenta pe oameni.

3 Server web

Câte servere pentru un vârf de trafic? Cât așteaptă un utilizator? Testul „pe viu” poate costa clienți.

4 Vremea

Plouă mâine în Timișoara? Atmosfera nu poate fi „rulată” înainte – doar modelată.

Ce au în comun?

- **Multe componente** care interacționează: mașini, oameni, cereri, mase de aer
- **Aleatorul** joacă un rol: sosiri, contacte, defecte
- **Evoluție în timp**: ne interesează ce se întâmplă mâine, peste o oră, la vârf
- **Experimentul direct** este imposibil, scump, lent sau periculos
- Vrem să răspundem la întrebări **„ce-ar fi dacă…?”**

Ideea cheie

Construim o **copie simplificată** a sistemului în calculator și experimentăm pe ea, nu pe sistemul real.

Ce este un sistem?

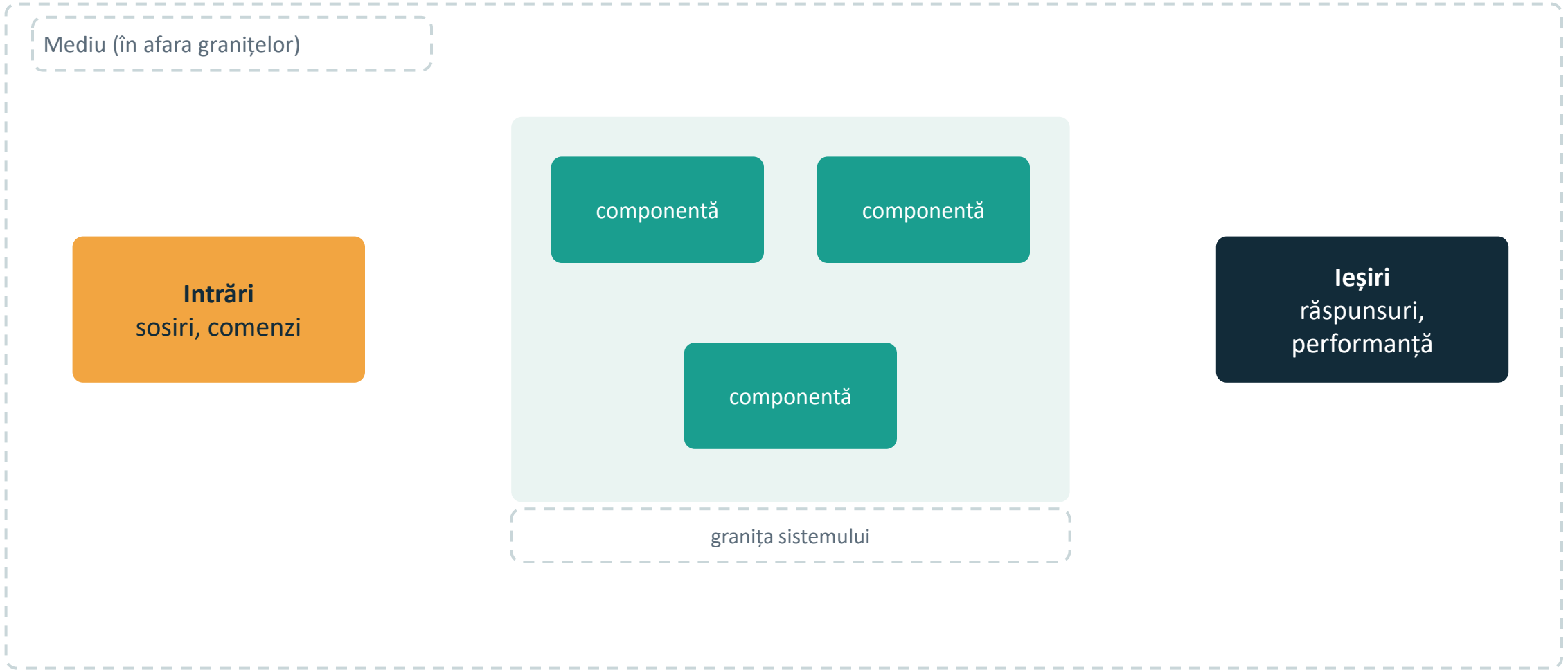
Sistem

O mulțime de **componente** care interacționează între ele și cu **mediul** înconjurător, pentru a îndeplini o funcție.

după Holban, cap. 1

- Exemple: o bancă, un procesor, o rețea de calculatoare, un oraș, un organism
- Granița sistemului o alegem **noi**, în funcție de întrebarea pe care o avem

Anatomia unui sistem



Vocabularul de bază

Concept	Definiție	Exemplu: ghișeu bancar
Entitate	un obiect al sistemului care ne interesează	clientul, casierul
Atribut	o proprietate a unei entități	ora sosirii, suma de plătit
Stare	mulțimea variabilelor care descriu sistemul la un moment dat	câți clienți sunt la coadă; casier liber/ocupat
Eveniment	o întâmplare instantanee care schimbă starea	sosește un client; se termină o plată
Activitate	o acțiune cu durată, între două evenimente	servirea unui client (3 minute)

Starea sistemului

Stare (state)

Informația **minimă** care, împreună cu intrările viitoare, determină complet evoluția viitoare a sistemului.

- La ghișeu: numărul de clienți la coadă și dacă funcționarul este ocupat
- Istoria detaliată (cine a venit la 9:03) nu mai este necesară pentru viitor
- Alegerea variabilelor de stare este prima decizie importantă de modelare

Evenimente și activități

Eveniment

- are loc într-un **moment** de timp
- schimbă **instantaneu** starea
- ex.: „sosire client” – coada crește cu 1
- ex.: „plecare client” – coada scade cu 1

vs

Activitate

- are o **durată**
- începe și se termină cu câte un eveniment
- ex.: „servire” – de la începutul la finalul plății
- durata poate fi fixă sau **aleatoare**

Același vocabular, un sistem informatic

Concept	Ghișeu bancar	Server web
Entitate	client	cerere HTTP
Atribut	ora sosirii, tipul operației	momentul sosirii, dimensiunea răspunsului
Resursă	casier	nucleu de procesor, conexiune la baza de date
Stare	clienți la coadă, casier ocupat	cereri în așteptare, thread-uri ocupate
Eveniment	sosire, început servire, plecare	sosire cerere, început procesare, răspuns trimis
Măsură de performanță	timpul mediu de așteptare	latența (ex. percentila 95), throughput

Exercițiu: cantina studențească

Descrieți în 3 minute, cu vecinul de bancă, sistemul „cantina la prânz”:

1 Entități și atribute

Cine „circulă” prin sistem? Ce proprietăți contează pentru întrebarea noastră?

2 Resurse

Ce resurse sunt limitate și pot crea cozi?

3 Stare

Ce variabile descriu cantina la un moment dat?

4 Evenimente

Ce întâmplări schimbă starea? Care sunt activitățile, cu durată?

Verificare

? Într-un model al unei parcări, care dintre următoarele este o **variabilă de stare**?

A Numărul de locuri ocupate în acest moment

B Culoarea mașinilor

C Numele proprietarului parcării

D Prețul unui bilet de tramvai

Ce este un model?

Model

O descriere **abstractă și simplificată** a unui sistem, construită pentru a răspunde la o **anumită întrebare**.

după Holban, cap. 1; Cernazean, curs 1

- Păstrează doar elementele **esențiale** pentru întrebarea pusă
- Poate fi scris ca ecuații, diagrame, reguli sau cod
- Nu se identifică niciodată cu sistemul pe care îl descrie

Ceci n’est pas une pipe. („Aceasta nu este o pipă.”)

— René Magritte, 1929

Pictura unei pipe nu este o pipă – la fel cum un model al traficului nu este traficul. „Harta nu este teritoriul.” (A. Korzybski)





All models are wrong, but some are useful.

— George E. P. Box, statistician

Întrebarea corectă nu este „este modelul adevărat?”, ci **„este suficient de bun pentru ce am nevoie?”**

La ce folosim modelele?

1 Descriere

Organizăm ce știm despre sistem: componente, relații, reguli.

2 Înțelegere

Aflăm **de ce** se comportă sistemul așa – de ex. de ce apar ambuteiaje „din nimic”.

3 Predicție

Estimăm ce se va întâmpla: câți bolnavi peste două săptămâni, latența la vârf.

4 Control și optimizare

Alegem cea mai bună variantă: câte servere, ce durată la semafor.

Ce este un model bun?

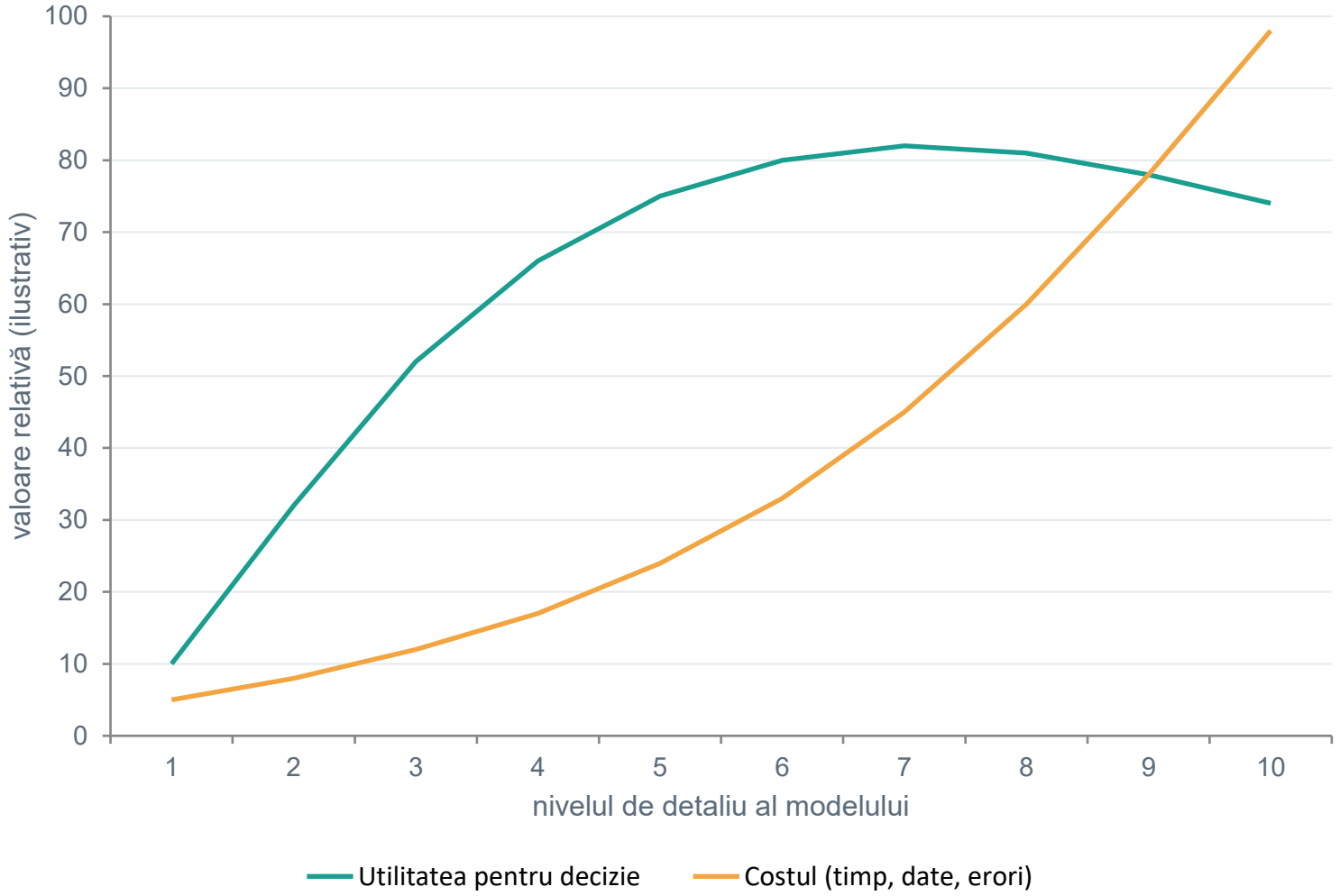
- Depinde de **întrebare**: același sistem poate avea mai multe modele bune
- Suficient de **simplu** ca să fie înțeles, implementat și verificat
- Suficient de **detaliat** ca să surprindă ce contează pentru decizie
- **Validat**: rezultatele se potrivesc cu realitatea acolo unde o cunoaștem
- **Transparent**: ipotezele sunt scrise explicit

E **Principiul simplității**

„Totul ar trebui făcut cât mai simplu posibil, dar nu mai simplu de atât.”

(atribuit lui A. Einstein)

Compromisul detaliu – cost



- La început, fiecare detaliu adăugat aduce mult
- Apoi câștigul scade, iar costul crește rapid
- Un model prea detaliat cere date pe care nu le avem și devine greu de verificat
- Zona bună este la mijloc

Același sistem, trei modele

Traficul pe un bulevard poate fi modelat la niveluri diferite:

1 Flux continuu

Traficul ca un **fluid**: densitate și viteză descrise prin ecuații. Bun pentru autostrăzi lungi.

2 Automat celular

Drumul împărțit în **celule**; fiecare mașină avansează după reguli simple. Arată cum apar ambuteiajele.

3 Agenți

Fiecare șofer este un **agent** cu destinație și comportament propriu. Bun pentru rute și decizii.

Ce este simularea?

Simulare

Tehnica de a realiza **experimente cu ajutorul calculatorului**, folosind un model care descrie comportamentul unui sistem real **pe un interval de timp**.

după Holban, cap. 1

- Simularea **imită** evoluția sistemului, pas cu pas sau eveniment cu eveniment
- Rezultatul: „instantanee” ale sistemului în diverse momente și condiții

Model vs simulare

Modelul

- este **descrierea** sistemului
- ecuații, reguli, cod
- ca o **rețetă** de bucătărie
- se construiește o dată, se verifică și se validează

vs

Simularea

- este **execuția** modelului în timp
- o rulare cu anumite date de intrare
- ca **gătitul** după rețetă
- se repetă de multe ori, în scenarii diferite

Experimente „in silico”

in vivo

experiment pe organismul / sistemul **real**

in vitro

experiment **în laborator**, în condiții controlate

in silico

experiment **în calculator** (siliciu) – pe un model

- Calculatorul devine un **laborator virtual**: putem repeta, opri, accelera și modifica experimentul
- Știința computațională: al treilea pilon, alături de teorie și experiment

Când folosim simularea?

1 Nu există soluție analitică

Sistemul e prea complex pentru formule: multe interacțiuni, reguli, aleator.

2 Experimentul real e imposibil

Prea scump, periculos sau lent: un oraș nou, o pandemie, un reactor.

3 Vrem să comparăm variante

Întrebări „ce-ar fi dacă…”: 3 servere sau 4? Semafor de 60 s sau 90 s?

4 Timpul trebuie comprimat

Ani de funcționare simulați în câteva secunde; sau fenomene foarte rapide „încetinite”.

5 Sistemul nu există încă

Proiectăm o fabrică, un procesor sau o rețea înainte de a le construi.

6 Instruire

Simulatoare de zbor, de conducere, de intervenție – învățare fără risc.

Exemplu din informatică: procesorul simulat

ani

durează proiectarea unui procesor nou – fabricarea unui prototip costă enorm

gem5

simulator open-source de arhitecturi: rulează programe reale pe un procesor „virtual”

ciclu cu ciclu

se simulează pipeline-ul, memoriile cache, predicția de salt

- Arhitecții compară variante (dimensiunea cache-ului, numărul de nuclee) **înainte** de a construi ceva
- Simularea este de mii de ori mai lentă decât hardware-ul real – dar mult mai ieftină decât un prototip greșit

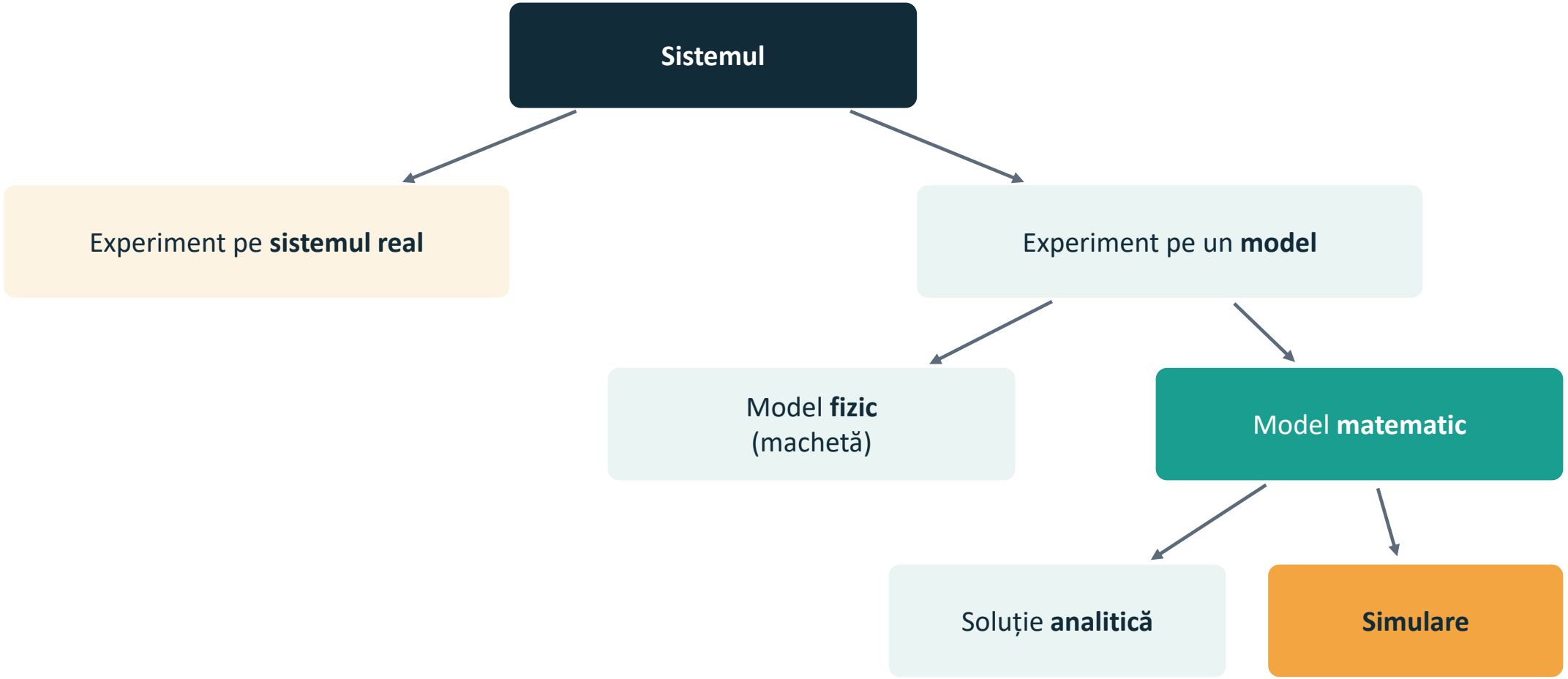
Când NU folosim simularea?

- Problema se rezolvă cu **bun-simț** sau cu o formulă simplă
- Este mai ieftin să **experimentăm direct** pe sistemul real
- Costul studiului **depășește** beneficiul deciziei
- Nu avem **date** și nici nu le putem estima rezonabil
- Nu avem timp sau oameni pentru **verificare și validare**
- Se așteaptă de la simulare **rezultate imposibile** (certitudini, „optimul garantat”)

Atenție

Simularea este un instrument **scump**. Uneori cel mai bun model este un calcul pe o foaie de hârtie.

Cum putem studia un sistem



Adaptat după A. M. Law, Simulation Modeling and Analysis

Soluție analitică vs simulare

Criteriu	Soluție analitică	Simulare
Rezultat	exact (formulă)	estimat, cu o marjă de eroare
Complexitate acceptată	mică – ipoteze restrictive	mare – aproape orice reguli
Efort de calcul	foarte mic	de la mic la foarte mare
Înțelegere	arată direct cum depinde rezultatul de parametri	cere multe rulări și analiză
Aleator	tratat prin probabilități	generat explicit, rulări repetate

Un exemplu de soluție analitică

$$W = 1 / (\mu - \lambda)$$

W	timpul mediu petrecut în sistem (așteptare + servire)
λ	rata medie de sosire (de ex. 8 clienți pe oră)
μ	rata medie de servire (de ex. 10 clienți pe oră)

Pentru λ = 8 și μ = 10: W = 1/2 oră = **30 de minute**. Valabilă doar pentru coada M/M/1 (sosiri Poisson, serviri exponențiale, un singur ghișeu) – o vom demonstra în săptămâna 4.

Când formula nu mai ajunge

Formula M/M/1 presupune

- un singur ghișeu
- sosiri complet aleatoare, cu rată constantă
- durate de servire exponențiale
- clienții nu pleacă de la coadă
- regim staționar (după mult timp)

Banca reală are

- 3 ghișee, unul doar pentru firme
- vârf de sosiri la prânz
- operații scurte și lungi
- clienți care renunță dacă e coadă mare
- program de 8 ore, pornește de la zero

Ce NU face simularea

- Nu găsește **singură** soluția optimă – evaluează variantele pe care i le dăm
- Nu oferă **certitudini** – oferă estimări, cu marjă de eroare
- Nu corectează un **model greșit** – „garbage in, garbage out”
- Nu înlocuiește **înțelegerea** sistemului – o cere

H **Holban, cap. 1**

Simularea nu oferă soluția optimă a problemei de decizie, ci doar **tabloul rezultatelor** care s-ar obține pentru variantele analizate.

O scurtă istorie a simulării

Buffon: estimarea lui π aruncând un ac – primul „experiment Monte Carlo”

1777

1946

Ulam, von Neumann, Metropolis: metoda Monte Carlo, pe calculatorul ENIAC

GPSS (G. Gordon, IBM): primul limbaj popular de simulare

1961

1967

Simula 67 (Dahl & Nygaard): primul limbaj orientat pe obiecte

Simulare în Python (**SimPy**), AnyLogic; simulare pe GPU

2000+

2024

GenCast: prognoză meteo cu AI, mai bună decât modelele clasice pe majoritatea țintelor

Curiozitate: OOP s-a născut din simulare

Simula

limbaj creat în Norvegia pentru **simularea** sistemelor (Dahl & Nygaard, 1962–1967)

clase

entitățile simulate au devenit **obiecte**, tipurile lor – **clase**

C++, Java

au preluat ideile din Simula (Stroustrup a lucrat cu Simula)

- Concepte precum **clasă**, **obiect**, **moștenire** au apărut pentru a descrie entitățile unui sistem simulat

Verificare

? În care situație simularea este **cea mai puțin** potrivită?

A Dimensionarea unei secții de urgențe care încă nu există

B Calculul dobânzii simple pentru un depozit bancar

C Evaluarea efectului unui nou sens giratoriu

D Estimarea latenței unui server la un vârf de trafic

Verificare

? Care afirmație descrie corect relația dintre **model** și **simulare**?

A Sunt sinonime

B Simularea este execuția unui model în timp, pentru a-i observa comportamentul

C Modelul este rezultatul unei simulări

D Simularea se folosește doar pentru modele deterministe

Rezumatul orei 1

- ✓ Simulăm când experimentul real este **imposibil, scump, lent sau periculos**
- ✓ Un **sistem** = componente care interacționează, cu o graniță aleasă de noi
- ✓ Vocabular: **entitate, atribut, stare, eveniment, activitate**
- ✓ Un **model** = descriere simplificată, construită pentru o întrebare
- ✓ O **simulare** = execuția modelului în timp – un experiment „in silico”
- ✓ Simularea dă **estimări**, nu certitudini, și nu optimizează singură

Ora 2

Ce fel de modele există?

1. Modele fizice, matematice, computaționale
2. Static / dinamic, determinist / stocastic
3. Timp și stare: continuu sau discret
4. Scale, emergență și harta metodelor din curs

Unde am rămas

- ✓ **Sistem**: componente + interacțiuni + graniță + mediu
- ✓ **Model**: descriere simplificată pentru o întrebare anume
- ✓ **Simulare**: execuția modelului în timp, ca experiment pe calculator

De ce clasificăm modelele?

- Tipul modelului determină **metoda** de rezolvare și **uneltele**
- Ne ajută să punem întrebările potrivite: contează timpul? contează aleatorul? contează spațiul?
- Ne ajută să comunicăm: „un model stocastic, cu evenimente discrete” spune mult în patru cuvinte
- Aceeași realitate poate fi descrisă cu modele de tipuri diferite

✓ **Pentru examen**

Veți primi descrieri scurte de modele și va trebui să le **clasificați** pe fiecare dintre axele din această oră.

Fizic, matematic, computațional

1 Model fizic

O **machetă** sau un prototip: aripă în tunel aerodinamic, macheta unui golf pentru studiul mareelor (Bay Model, 1957).

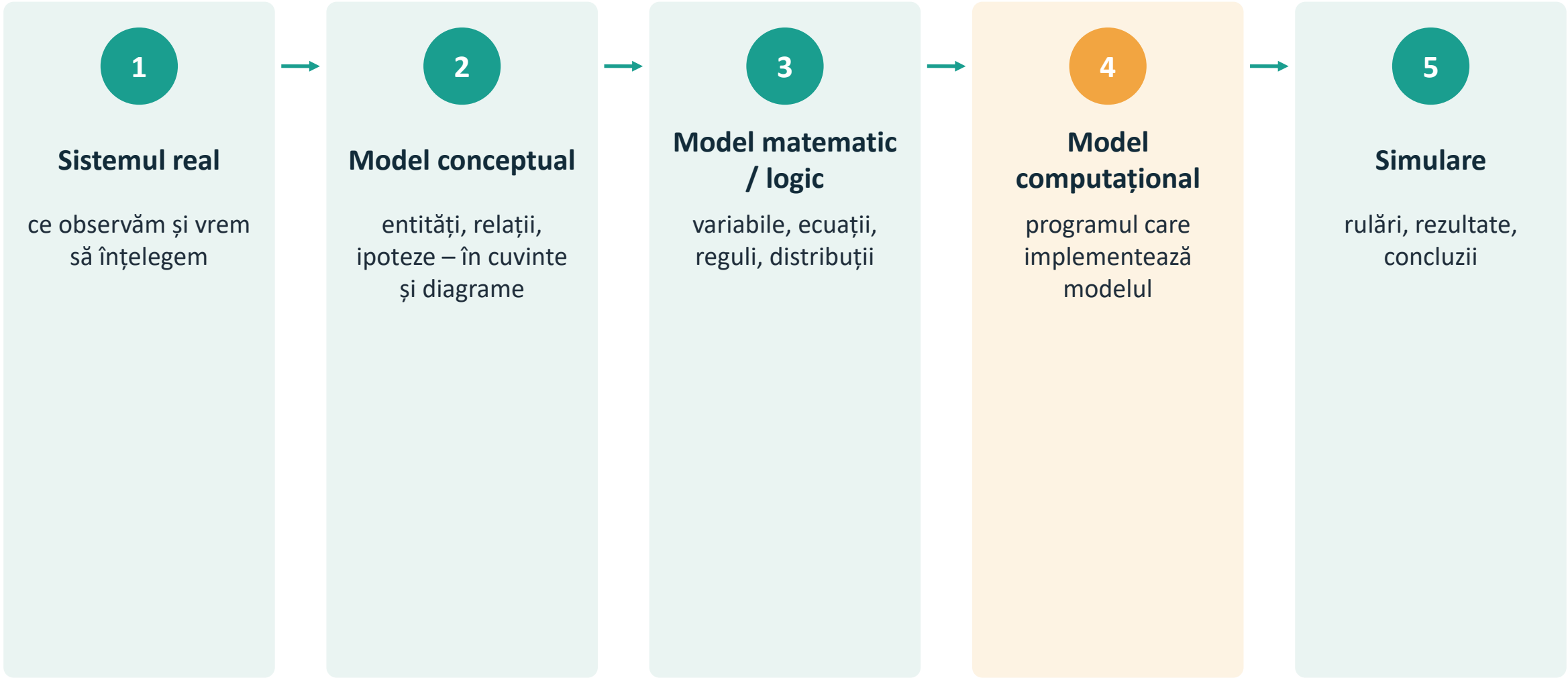
2 Model matematic

Ecuații și relații: legea lui Newton, formula cozii M/M/1, ecuațiile unei epidemii.

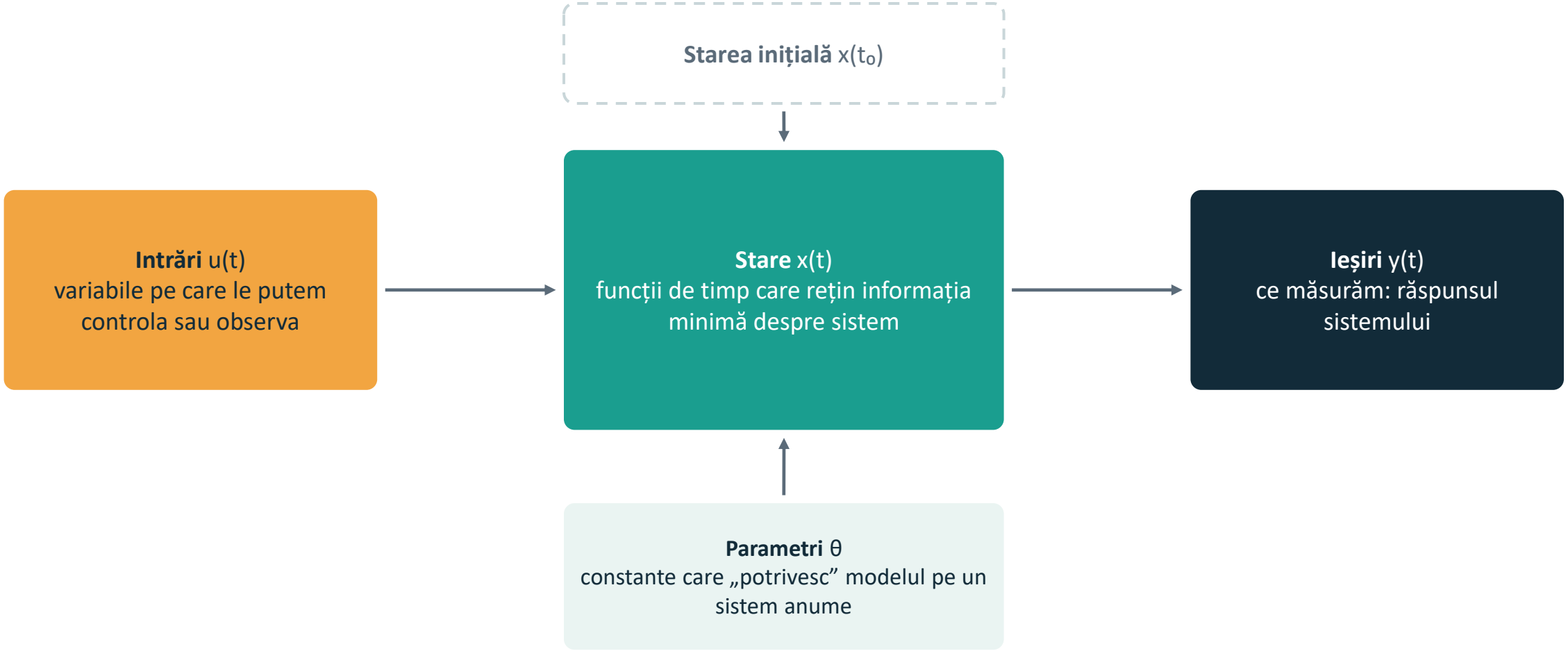
3 Model computațional

Un **program** care implementează reguli: agenți, evenimente, celule. Adesea fără soluție analitică.

De la idee la program



Elementele unui model matematic



Static vs dinamic

Model static

- timpul **nu** joacă un rol
- descrie sistemul într-un moment sau „în medie”
- ex.: riscul unui proiect estimat din 10 000 de scenarii aleatoare
- ex.: calculul rezistenței unei grinzi

vs

Model dinamic

- descrie **evoluția în timp**
- starea se schimbă de la un moment la altul
- ex.: coada de la ghișeu pe parcursul unei zile
- ex.: numărul de infectați săptămână de săptămână

Determinist vs stocastic

Model determinist

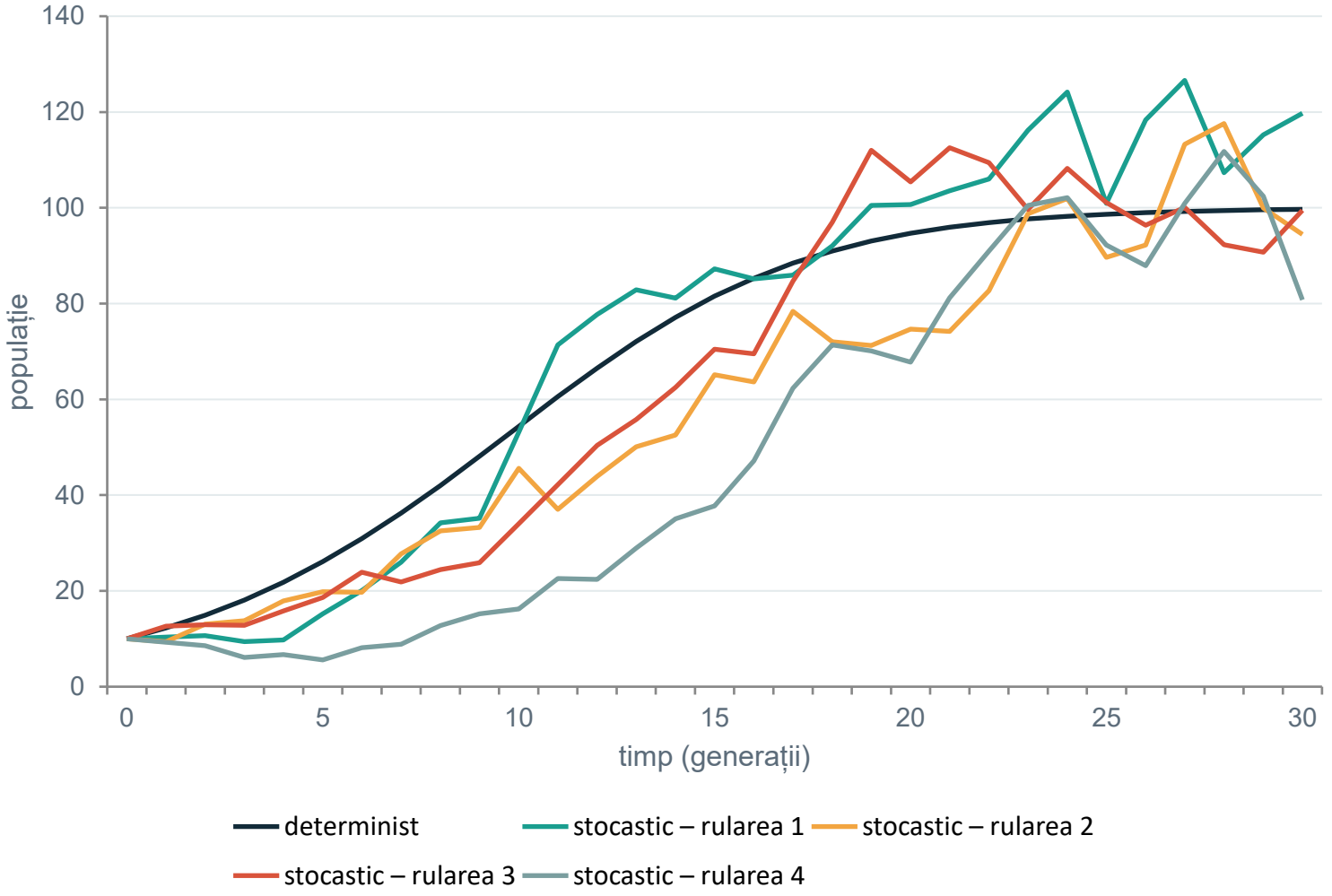
- **fără** elemente aleatoare
- aceleași date de intrare → **același** rezultat
- o singură rulare este suficientă
- ex.: traiectoria unei mingi, un circuit RC

vs

Model stocastic

- conține **variabile aleatoare**
- aceleași date de intrare → rezultate **diferite**
- avem nevoie de **multe rulări** și de statistică
- ex.: sosirile la ghișeu, defectele unor servere

Aceeași ecuație, cu și fără aleator



- Linia închisă: modelul **determinist** – o singură curbă
- Liniile colorate: **patru rulări** ale aceluiași model cu zgomot aleator
- Media rulărilor seamănă cu curba deterministă, dar fiecare rulare e diferită

De ce contează aleatorul?

1

rulare a unui model stocastic **nu dovedește nimic**

30+

rulări independente: un punct de pornire uzual pentru statistici

±

orice rezultat se raportează cu un **interval de încredere**

- Variabilitatea nu este o „eroare” a simulării – este o proprietate a **sistemului real**
- Media nu spune tot: la servere contează și **cazurile extreme** (percentila 95, 99)

Timp continuu vs timp discret

Timp continuu

- starea poate fi observată **în orice moment** t
- descris de obicei prin **ecuații diferențiale**
- ex.: temperatura unei cafele care se răcește
- ex.: poziția unei planete

vs

Timp discret

- starea este definită doar la momente **t = 0, 1, 2, …**
- descris prin **recurențe**: x(t+1) = f(x(t))
- ex.: populația de la un an la altul
- ex.: soldul unui cont la sfârșit de lună

Stare continuă vs stare discretă

Stare continuă

- variabilele de stare iau **valori reale**
- se schimbă lin, gradual
- ex.: temperatura, nivelul apei, concentrația
- ex.: fracțiunea de populație infectată

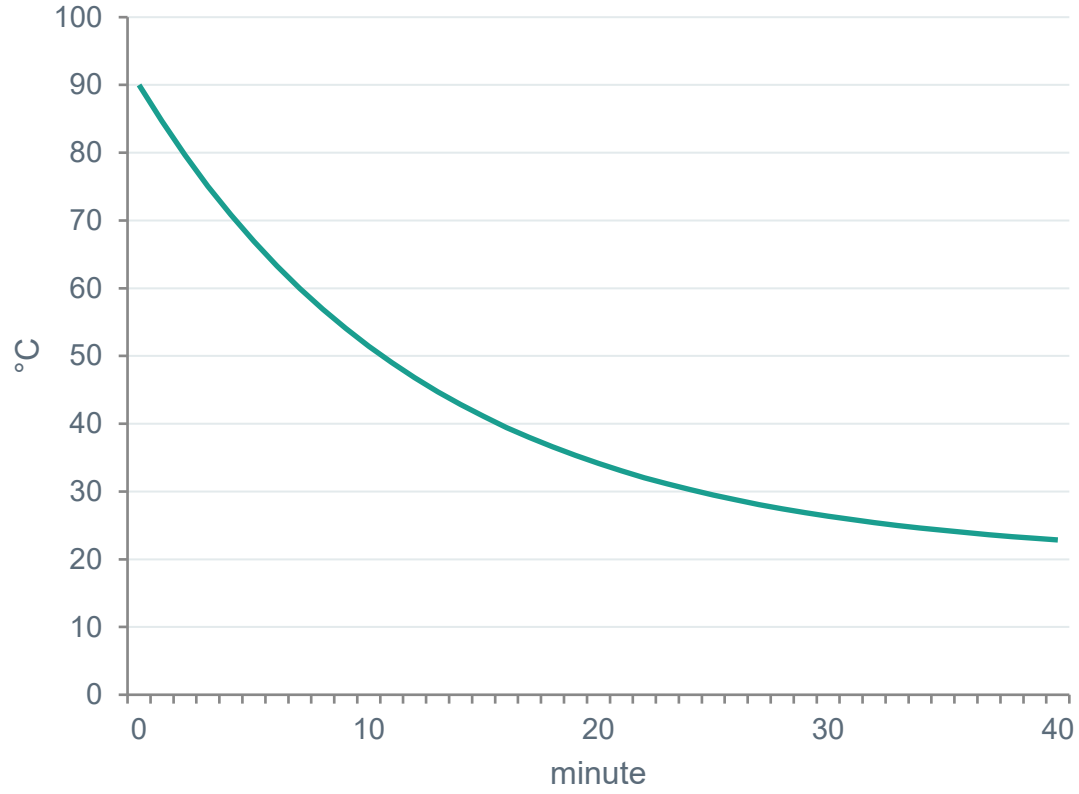
vs

Stare discretă

- variabilele iau **valori numărabile**
- se schimbă prin **salturi**
- ex.: numărul de clienți la coadă (0, 1, 2, …)
- ex.: server pornit / oprit; semafor roșu / verde

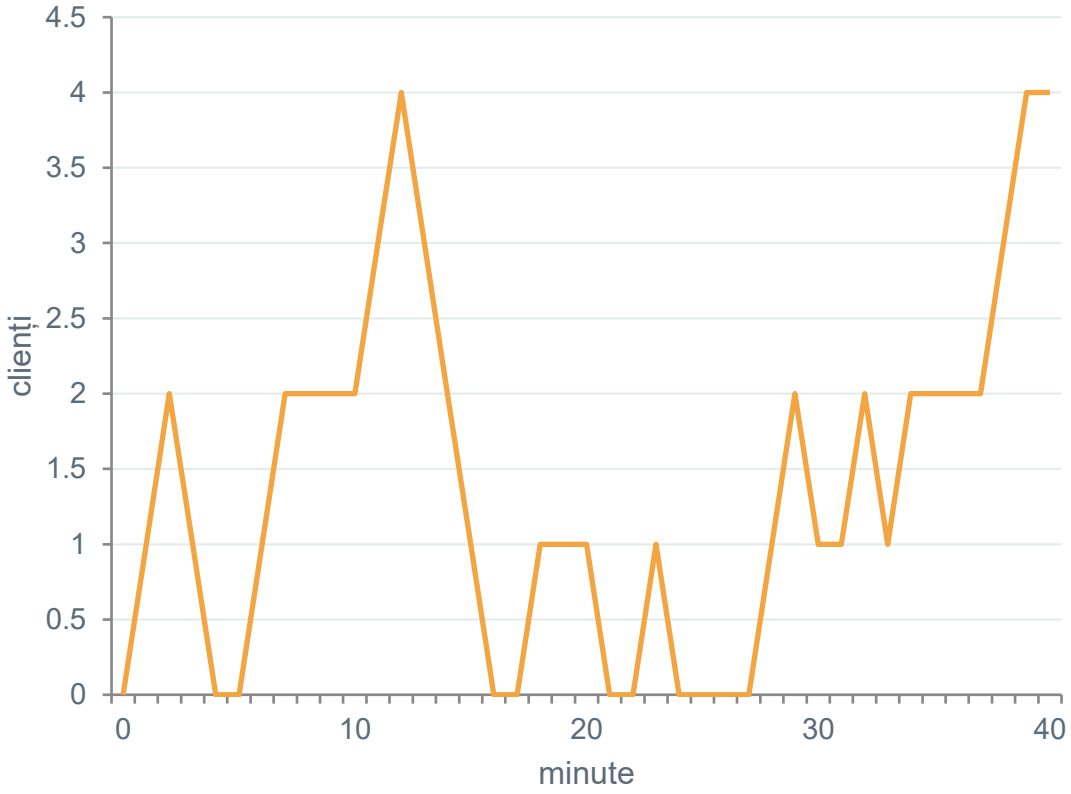
Continuu vs discret, vizual

Temperatura cafelei (stare continuă)



Se schimbă **lin**, în orice moment

Clienți la coadă (stare discretă)



Se schimbă prin **salturi** întregi

Timp × stare: patru familii de modele

	Stare continuă	Stare discretă
Timp continuu	**Ecuații diferențiale (EDO)** răcire, epidemii SIR, pradă–prădător, circuite. **Săptămâna 7**	**Evenimente discrete (DES)** cozi, fabrici, rețele: starea sare în momente arbitrare. **Săptămânile 4–5**
Timp discret	**Recurențe / ecuații cu diferențe** populații anuale, modele economice lunare, sisteme eșantionate.	**Automate celulare, lanțuri Markov** celule și stări finite, pași de timp ficși. **Săptămânile 3 și 8**

Discret sau continuu? E alegerea modelatorului

- Traficul pe autostradă: **fluid continuu** (densitate, viteză) sau **mașini individuale** (discret)
- O populație de milioane: **număr real** în ecuații sau **indivizi** numărați unul câte unul
- Un server: **flux de cereri** (continuu) sau **fiecare cerere** ca eveniment (discret)
- Regula practică: cu **număr mare** de entități, continuul e adesea o aproximare bună

Ideea cheie

Clasificarea se aplică **modelului**, nu sistemului. Același sistem poate fi modelat discret sau continuu, după întrebare.

Descriptiv, predictiv, prescriptiv

1 Descriptiv

Ce se întâmplă și de ce? Modele care explică: de ce apare coada la ora 12.

2 Predictiv

Ce se va întâmpla? Câți utilizatori vor fi mâine, câți bolnavi peste o lună.

3 Prescriptiv

Ce ar trebui să facem? Câte servere, ce program, ce politică – combinat cu optimizare.

Același sistem, trei întrebări

Tip	Întrebare pentru un sistem de biciclete partajate	Ce rezultat așteptăm
Descriptiv	De ce stația din campus rămâne fără biciclete dimineața?	o explicație: fluxul spre campus e mai mare
Predictiv	Câte biciclete vor fi la fiecare stație la ora 9?	o estimare, cu marjă de eroare
Prescriptiv	Câte biciclete să mutăm noaptea și unde?	o recomandare de politică de rebalansare

Top-down vs bottom-up

Top-down (agregat)

- descriem **mărimi globale**: câți infectați, ce densitate
- ecuații pentru medii
- rapid, puțini parametri
- presupune o populație **omogenă**

vs

Bottom-up (individual)

- descriem **fiecare individ** și regulile lui
- comportamentul global **rezultă** din interacțiuni
- costisitor, mulți parametri
- surprinde **diversitatea** și spațiul

Exemplu: o epidemie în două stiluri

```
# TOP-DOWN: ecuații pentru fracțiuni (model SIR)
# dS/dt = -b*S*I
# dI/dt =  b*S*I - g*I
# dR/dt =  g*I

# BOTTOM-UP: fiecare persoană este un agent
for p in persoane:
    if p.stare == "infectat":
        for v in p.vecini():
            if v.stare == "susceptibil" and random() < b:
                v.stare = "infectat"
        if random() < g:
            p.stare = "vindecat"
```

- Sus: 3 ecuații, 2 parametri (b, g) – săptămâna 7
- Jos: reguli pentru indivizi; cine se întâlnește cu cine contează – săptămâna 9
- Ambele pot reproduce aceeași curbă „în medie”

Emergența

Comportament emergent

Un tipar **global** care apare din **interacțiuni locale** simple, fără să fie programat explicit în regulile indivizilor.

- Stolurile de păsări: fiecare pasăre urmează 3 reguli simple, stolul pare „coordonat”
- Ambuteiajul fantomă: o frânare mică se propagă înapoi și creează un blocaj
- Motivul pentru care modelele bottom-up pot surprinde fenomene pe care ecuațiile nu le arată

Niveluri de realitate



Același sistem poate fi descris la scări diferite; metoda depinde de scara de interes (Cernazean, curs 1)

Scări în exemple concrete

Domeniu	Micro	Meso	Macro
Fluide	molecule	element de fluid, rețea Boltzmann	câmp de presiune, vreme
Biologie	proteine, celule	țesut	organ, organism
Trafic	mașina și șoferul	pluton de mașini	flux pe bulevard, oraș
Rețele	pachet IP	fluxuri TCP	încărcarea rețelei

Modelăm adesea mai fin decât întrebarea

- Întrebarea este **macro** (se formează ambuteiaj?), modelul este **micro** (reguli pentru fiecare mașină)
- Detaliile microscopice contează adesea **puțin** pentru comportamentul macroscopic
- Contează în schimb **legile de conservare** și **simetriile**: nicio mașină nu dispare, nicio masă nu se pierde
- Căutăm cel mai simplu model micro care reproduce comportamentul macro corect

Exemplu

Un automat celular cu reguli foarte simple poate reproduce curgerea unui fluid, dacă respectă conservarea masei și a impulsului (săptămâna 8).

Spațiul și timpul în modele

1 Doar timp

Ne interesează evoluția unei mărimi globale, **indiferent de loc**: numărul total de clienți, de infectați.

2 Timp și spațiu

Mărimea diferă de la loc la loc și se schimbă în timp: temperatura atmosferei, poziția mașinilor.

3 Doar spațiu

Proces **staționar**: temperatura într-o cameră în regim stabil, lângă geam vs la mijloc.

Scara timpului: de la nanosecunde la decenii

ns

un ciclu de procesor, o instrucțiune în pipeline

ms

o cerere HTTP, un pachet în rețea

min

un client la ghișeu, o mașină la semafor

ani

o epidemie, clima, uzura unei infrastructuri

- Scara timpului decide **pasul** simulării și **orizontul** ei
- Un model nu poate acoperi eficient toate scările: alegem scara potrivită întrebării

Modele pe care le folosiți zilnic

1 Timpul estimat în Maps

Model predictiv al traficului, antrenat pe date și actualizat continuu.

2 Prognoza meteo

Simulări numerice ale atmosferei și, mai nou, modele AI antrenate pe simulări.

3 Motorul fizic dintr-un joc

Simulare de corpuri rigide, coliziuni, particule – de 60 de ori pe secundă.

4 Teste de încărcare

Utilizatori virtuali care „bombardează” o aplicație înainte de lansare.

Verificare

? Estimăm riscul unui proiect generând **10 000 de scenarii aleatoare** pentru costuri și calculând media. Modelul este:

A static și stocastic

B dinamic și determinist

C dinamic și stocastic

D static și determinist

Verificare

? Simulăm o coadă la ghișeu **eveniment cu eveniment**: sosiri și plecări aleatoare, la momente oarecare. Starea (numărul de clienți) este:

A continuă, în timp discret

B discretă, în timp continuu

C continuă, în timp continuu

D discretă, în timp discret

Verificare

? Traiectoria unei mingi aruncate, calculată cu legile lui Newton, fără vânt și fără alte perturbații, este un model:

A static și stocastic

B dinamic, determinist, cu stare continuă

C dinamic, stocastic, cu stare discretă

D static și determinist

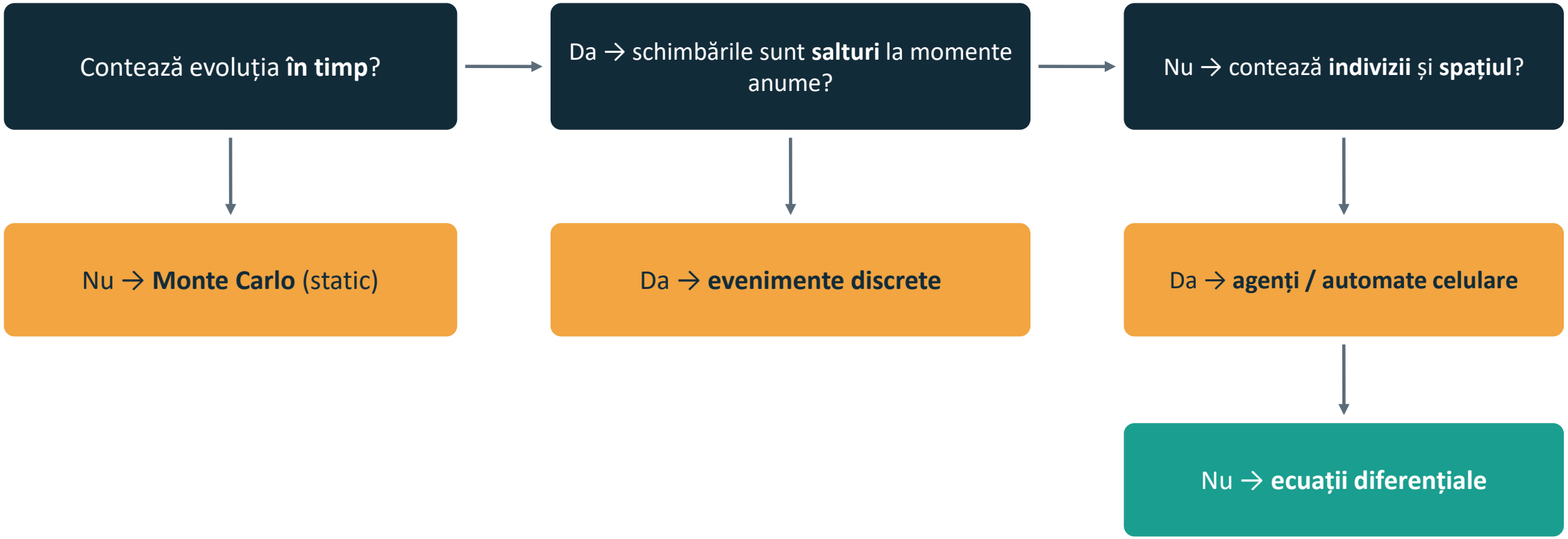
Exercițiu: clasificați modelele

Model	Static / dinamic	Det. / stoc.	Stare
Nivelul apei într-un rezervor, cu robinet și scurgere constante	?	?	?
Numărul de pachete în coada unui router, cu sosiri aleatoare	?	?	?
Profitul estimat al unui magazin din 5 000 de scenarii de vânzări	?	?	?
Game of Life: celule vii / moarte pe o grilă, reguli fixe	?	?	?

Harta metodelor din curs

Metodă	Ideea în câteva cuvinte	Săpt.	Tip
Monte Carlo	estimăm prin eșantionare aleatoare repetată	S2	static, stocastic
Lanțuri Markov	stări și probabilități de tranziție	S3	discret
Teoria cozilor	formule pentru sisteme de servire	S4	analitic
Evenimente discrete	starea sare la momentele evenimentelor	S5	dinamic, stocastic
Ecuații diferențiale	schimbări continue ale mărimilor agregate	S7	continuu
Automate celulare	celule cu reguli locale, pe o rețea	S8	discret, spațial
Modele cu agenți	indivizi autonomi care interacționează	S9	bottom-up
Modele din date / AI	modelul se învață din date sau din simulări	S10	data-driven

Cum aleg metoda? O primă schiță



O simplificare utilă pentru început; în practică metodele se combină

Proprietățile unui model bun

1 Valid

Reproduce realitatea acolo unde o cunoaștem, pentru întrebarea pusă.

2 Simplu

Nu conține nimic ce nu influențează răspunsul.

3 Transparent

Ipotezele și datele sunt scrise și pot fi verificate de alții.

4 Reproductibil

Alt om, cu același cod și aceleași date, obține aceleași rezultate.

5 Util

Răspunde la timp la o întrebare care contează pentru o decizie.

6 Robust

Concluziile nu se schimbă radical la variații mici ale parametrilor.

KISS vs KIDS – două filozofii

KISS: Keep It Simple, Stupid

- pornim de la **cel mai simplu** model posibil
- adăugăm detalii doar dacă sunt necesare
- ușor de înțeles, de verificat, de explicat
- risc: omitem ceva important

KIDS: Keep It Descriptive, Stupid

- pornim de la un model **bogat**, apropiat de date
- simplificăm doar ce se dovedește inutil
- mai realist, mai ușor de validat cu date
- risc: model greu de înțeles și de verificat

Greșeli frecvente de modelare

- Modelul **prea complicat** de la început – nimeni nu mai înțelege ce face
- Modelul **prea simplu** – lipsește tocmai mecanismul care contează
- Date de intrare **inventate** sau nepotrivite („garbage in, garbage out”)
- Concluzii dintr-o **singură rulare** a unui model stocastic
- Lipsa **verificării**: bug-uri care produc rezultate „plauzibile”
- Confundarea modelului cu **realitatea**

✓ **Sfat**

Scrieți ipotezele **înainte** de a scrie codul. Lista de ipoteze este cel mai valoros document al unui studiu.

Limbajele în care scriem modele

1 Cuvinte

Modelul conceptual: ce intră, ce ignorăm, ce presupunem.

2 Diagrame

Grafuri de stări, fluxuri, activități, stocuri.

3 Matematică

Ecuații, probabilități, distribuții.

4 Cod

Python, SimPy, Mesa, AnyLogic – modelul executabil.

Glosar – ora 2 (RO – EN)

Română	English	Pe scurt
model static / dinamic	static / dynamic model	fără / cu evoluție în timp
determinist / stocastic	deterministic / stochastic	fără / cu aleator
timp continuu / discret	continuous / discrete time	orice moment / pași
stare continuă / discretă	continuous / discrete state	valori reale / numărabile
descriptiv / predictiv / prescriptiv	descriptive / predictive / prescriptive	explică / prezice / recomandă
comportament emergent	emergent behavior	global, din reguli locale

Rezumatul orei 2

- ✓ Modele **fizice, matematice, computaționale**; drumul: conceptual → formal → cod
- ✓ **Static / dinamic**, **determinist / stocastic** – modelele stocastice cer multe rulări
- ✓ **Timp** și **stare**, fiecare continuu sau discret → patru familii de modele
- ✓ Discret sau continuu este o **alegere a modelatorului**
- ✓ **Top-down** vs **bottom-up**; **emergența** apare din interacțiuni locale
- ✓ Harta metodelor: Monte Carlo, Markov, cozi, DES, EDO, CA, agenți, AI

Ora 3

Cum se face un studiu de simulare

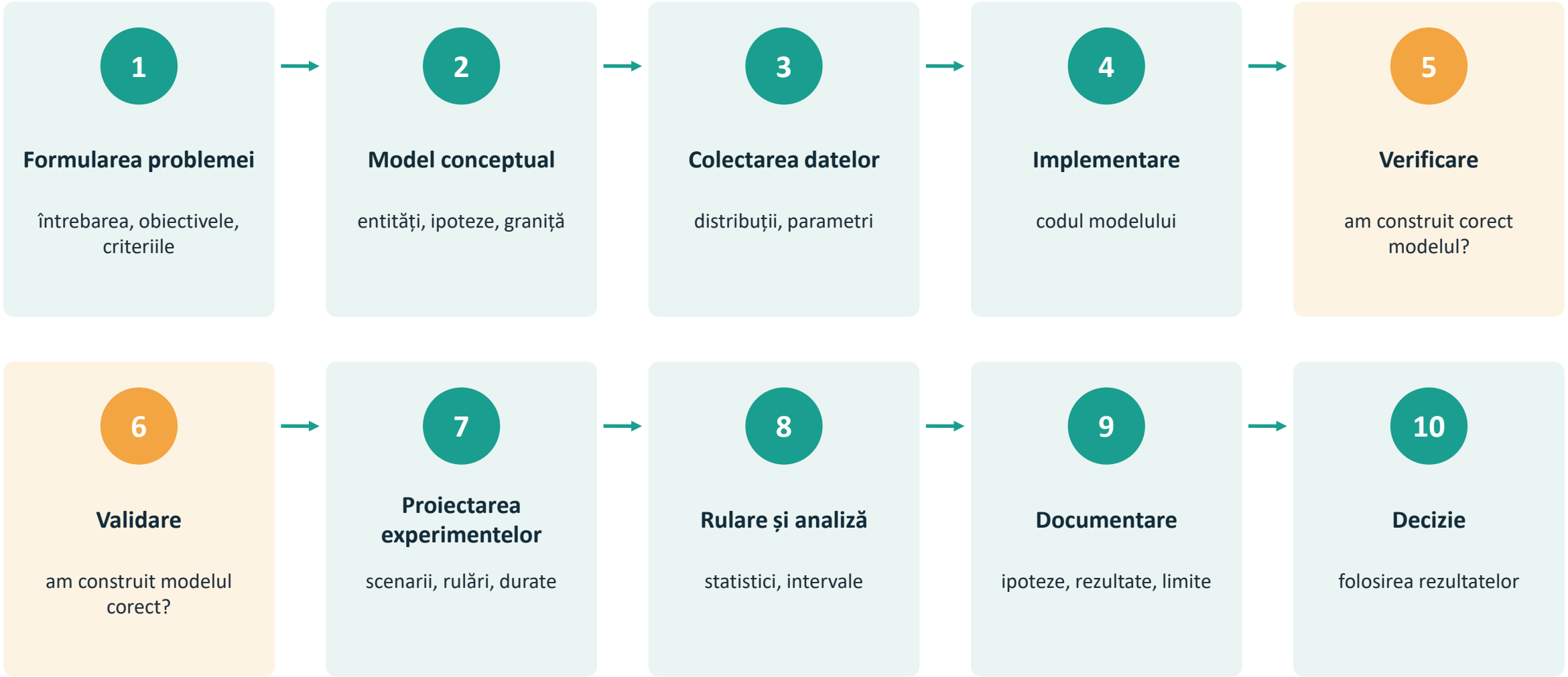
1. Cele zece etape ale unui studiu
2. Verificare și validare
3. Studiu de caz: biciclete partajate, în Python
4. Frontiera: unde se folosește simularea azi

Un studiu de simulare este un proiect, nu un program.

— o regulă de bază a meseriei

Codul este de obicei **partea cea mai mică**. Timpul se duce pe înțelegerea problemei, date, verificare, validare și explicarea rezultatelor.

Etapele unui studiu de simulare



Procesul este iterativ: ne întoarcem des la pașii anteriori. (adaptat după Law; Banks et al.; Holban 1.2)

1. Formularea problemei

- Care este **întrebarea**? Cine va folosi răspunsul și pentru ce decizie?
- Ce **măsuri de performanță** ne interesează? (timp de așteptare, cost, utilizare)
- Ce **alternative** comparăm? (3 vs 4 servere, două politici)
- Care este **orizontul de timp**? (o zi, un an, un vârf de trafic)
- Ce **precizie** este suficientă și până când avem nevoie de răspuns?

Cea mai frecventă greșeală

Să începi să modelezi înainte să știi **exact** ce întrebare ai. „Să simulăm fabrica” nu este o întrebare.

2. Modelul conceptual

- Ce **entități** și ce **resurse** există? Ce atribute contează?
- Care este **granița** sistemului? Ce considerăm mediu?
- Care sunt **evenimentele** și **regulile** de funcționare?
- Ce **ipoteze simplificatoare** facem – scrise explicit, pe listă
- Rezultatul: o descriere în cuvinte și diagrame, **validată cu experții** sistemului

? Test rapid

Poate un expert al sistemului (nu un programator) să citească modelul conceptual și să spună „da, așa funcționează”?

Ipoteze simplificatoare: exemplu

Păstrăm în model

- sosirile aleatoare ale clienților
- durata variabilă a servirii
- numărul de ghișee deschise
- vârful de la prânz

Ignorăm (deocamdată)

- clienții care vin în grup
- pauzele funcționarilor
- defectarea sistemului informatic
- diferențele dintre funcționari

3. Colectarea datelor

- Surse: **măsurători**, loguri de sistem, baze de date, experți, literatură
- Din date extragem **distribuții**: cât durează servirea? cât de des sosesc clienții?
- Atenție la date **incomplete**, vechi sau dintr-o perioadă atipică
- Când nu există date: estimări de la experți + **analiză de sensibilitate**
- Detalii despre modelarea datelor de intrare: **săptămâna 6**

G **Garbage in, garbage out**

Un model perfect, alimentat cu date proaste, produce rezultate proaste – cu mare precizie.

4. Implementarea: ce unelte folosim?

Tip de unealtă	Exemple	Când le alegem
Limbaj general	Python (NumPy, SciPy), C++, Java, Julia	flexibilitate maximă, modele noi, cercetare
Bibliotecă de simulare	SimPy (evenimente), Mesa (agenți)	structură gata făcută, tot în Python
Mediu specializat	AnyLogic, Arena, Simio, FlexSim	industrie, animație, modele mari, echipe mixte
Sisteme fizice	Modelica / OpenModelica, Simulink	sisteme tehnice: mecanice, electrice, termice
Agenți, educație	NetLogo	prototipuri rapide, modele didactice

5–6. Verificare vs validare

Verificare

- **„Am construit corect modelul?”**
- programul face ce spune modelul conceptual?
- caută **bug-uri** și erori de logică
- tehnici: teste, trace, cazuri extreme, comparare cu formule

vs

Validare

- **„Am construit modelul corect?”**
- modelul reprezintă suficient de bine **realitatea**?
- caută **ipoteze greșite**
- tehnici: comparare cu date reale, păreri ale experților

Tehnici de verificare și validare

1 Cazuri extreme

Rata de sosire zero → coada trebuie să fie goală. Un singur client → fără așteptare.

2 Comparare cu formule

În cazul simplu (M/M/1), simularea trebuie să dea aproape rezultatul teoretic.

3 Trace și animație

Urmărim pas cu pas câteva entități; animația arată imediat comportamente absurde.

4 Date istorice

Rulăm modelul pe condițiile de anul trecut și comparăm cu ce s-a întâmplat.

7–8. Experimente și analiză

- Ce **scenarii** rulăm? (ex.: 2, 3, 4, 5 servere × trafic normal / vârf)
- Câte **replicări** pentru fiecare scenariu? Cât de lungă este o rulare?
- Eliminăm **perioada de încălzire** când simulăm regimul staționar
- Raportăm **medii cu intervale de încredere**, nu numere singure
- Comparăm alternativele **statistic**, nu „din ochi”

6 **Săptămâna 6**

Vom vedea exact cum se calculează intervalele de încredere și câte replicări sunt necesare.

9–10. Documentare și decizie

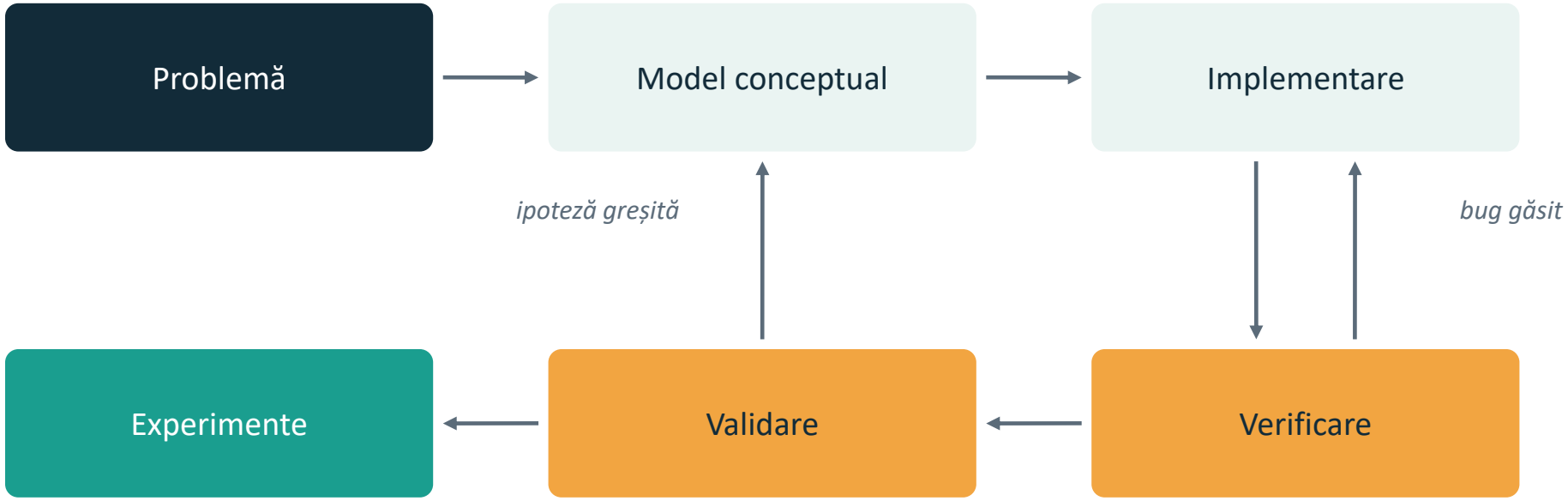
Documentare

- lista **ipotezelor** și a surselor de date
- versiunea codului și **seed-urile** folosite
- rezultatele și **limitele** lor
- ce **nu** poate răspunde modelul

Decizie

- rezultatele explicate pe limba **beneficiarului**
- grafice clare, nu tabele imense
- recomandare + incertitudinea ei
- uneori: modelul rămâne în uz și este **actualizat** (digital twin)

Procesul este iterativ



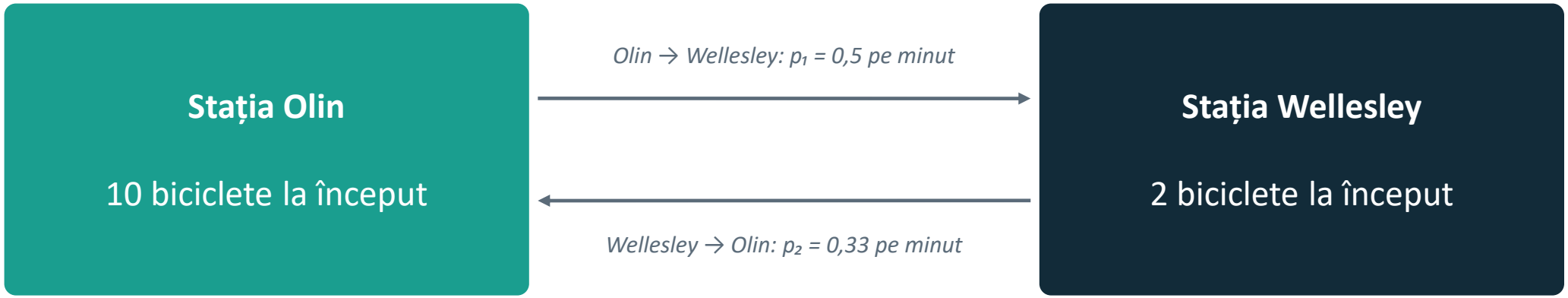
Verificarea ne trimite înapoi la cod; validarea ne trimite înapoi la ipoteze

Studiu de caz

Biciclete partajate între două campusuri

1. Problema și modelul conceptual
2. Implementarea în Python
3. Rezultate, variabilitate, experiment
4. Verificare și validare

Sistemul: două stații, 12 biciclete



Studenții iau o bicicletă de la o stație și o lasă la cealaltă. Dacă stația e goală, clientul pleacă nemulțumit.

După A. B. Downey, Modeling and Simulation in Python, cap. 2–3

Formularea problemei

- **Întrebare descriptivă**: cum evoluează numărul de biciclete la fiecare stație?
- **Măsura de performanță**: numărul de clienți nemulțumiți (au găsit stația goală)
- **Alternative**: cum influențează rata de plecări p_1 numărul de nemulțumiți?
- **Orizont**: o perioadă de 60 de minute, pas de 1 minut

Observați

Întrebarea este precisă și măsurabilă. De aici știm ce trebuie să înregistreze simularea.

Modelul conceptual

Element	În modelul bike share
Stare	numărul de biciclete la Olin și la Wellesley (suma este mereu 12)
Timp	discret: pași de câte 1 minut
Evenimente	plecare Olin → Wellesley (probabilitate p_1); plecare Wellesley → Olin (probabilitate p_2)
Regulă	dacă stația de plecare este goală, clientul nu primește bicicletă și e contorizat ca nemulțumit
Ipoteze	drumul durează neglijabil; cel mult o plecare în fiecare sens pe minut; p_1, p_2 constante
Tip de model	dinamic, stocastic, timp discret, stare discretă

Implementare: starea și pasul de timp

```
import random

# starea sistemului
stare = {"olin": 10, "wellesley": 2, "nemultumiti": 0}

def muta(stare, de_la, la):
    if stare[de_la] == 0:          # stație goală
        stare["nemultumiti"] += 1
        return
    stare[de_la] -= 1
    stare[la] += 1

def pas(stare, p1, p2):
    if random.random() < p1:
        muta(stare, "olin", "wellesley")
    if random.random() < p2:
        muta(stare, "wellesley", "olin")
```

- Starea: un dicționar cu 3 valori
- **muta** aplică regula stației goale
- **pas** = un minut: două „aruncări de monedă” cu probabilitățile p_1 și p_2
- random.random() întoarce un număr uniform în [0, 1)

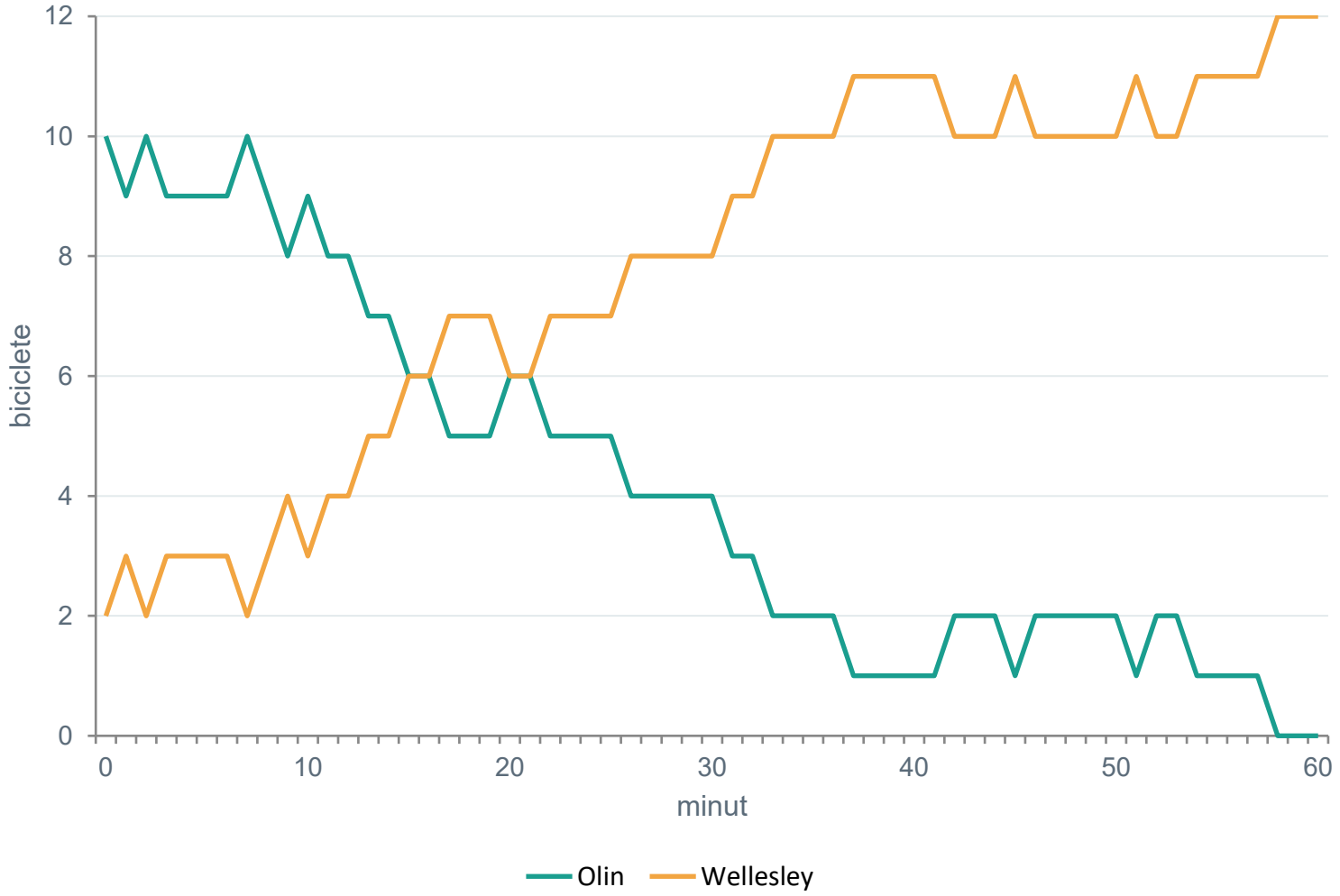
Implementare: bucla de simulare

```
def simuleaza(p1, p2, minute=60, seed=None):
    random.seed(seed)        # reproductibilitate
    stare = {"olin": 10, "wellesley": 2,
             "nemultumiti": 0}
    istoric = [stare["olin"]]
    for t in range(minute):
        pas(stare, p1, p2)
        istoric.append(stare["olin"])
    return istoric, stare["nemultumiti"]

istoric, nemultumiti = simuleaza(0.5, 0.33, seed=1)
```

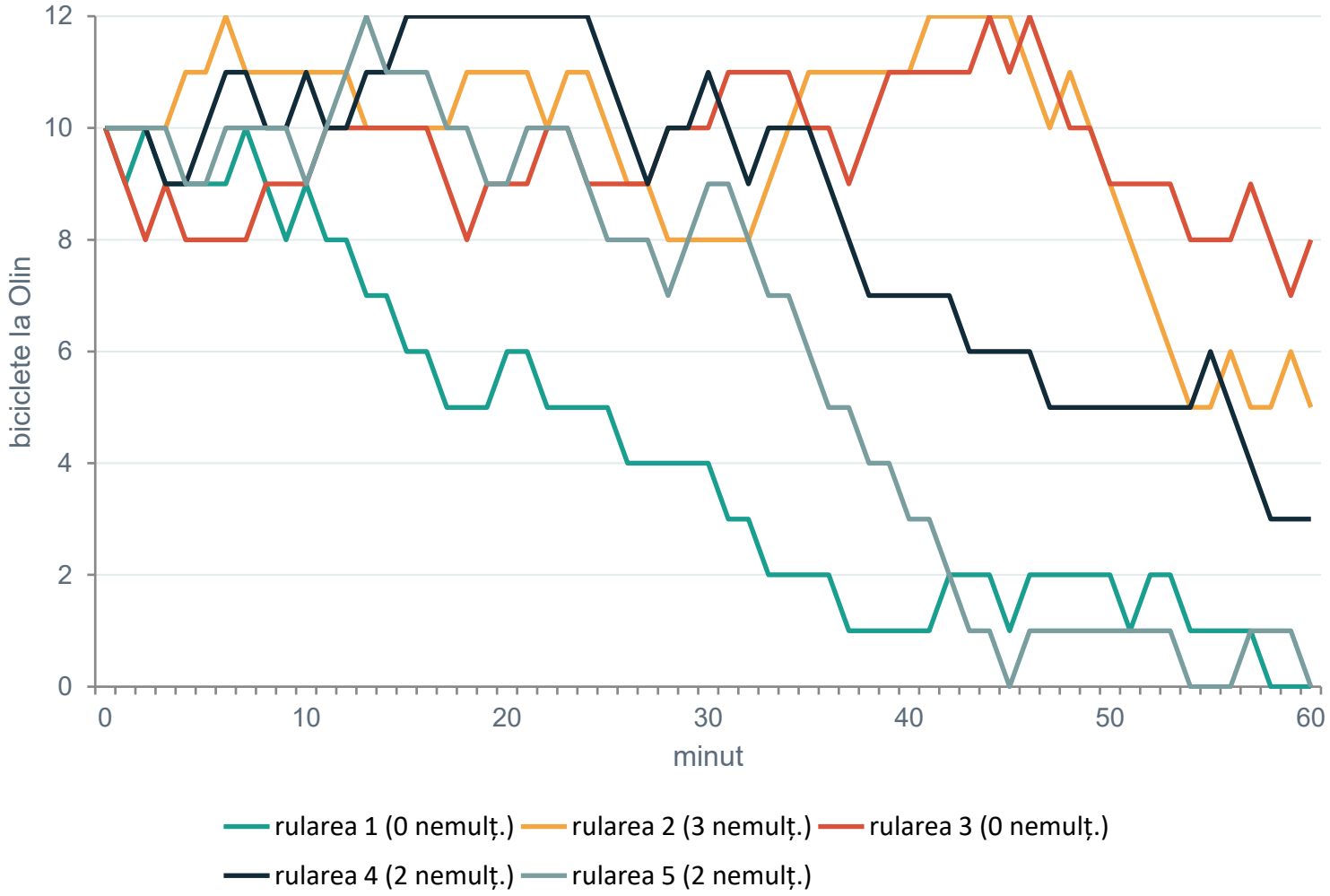
- Bucla avansează timpul minut cu minut
- Înregistrăm istoricul stării pentru grafic
- **seed** fix → aceeași rulare de fiecare dată (util la depanare)
- Rezultat: evoluția la Olin + nr. de nemulțumiți

O rulare: evoluția numărului de biciclete



- Suma rămâne mereu **12** – o primă verificare
- Bicicletele „curg” spre Wellesley, pentru că $p_1 > p_2$
- În această rulare: **0** clienți nemulțumiți

Cinci rulări: același model, rezultate diferite



- Modelul este **stocastic**: fiecare rulare e alta
- După 60 de minute Olin are între 0 și 8 biciclete
- Nemulțumiți: între 0 și 3
- Concluziile cer **multe rulări** și statistică

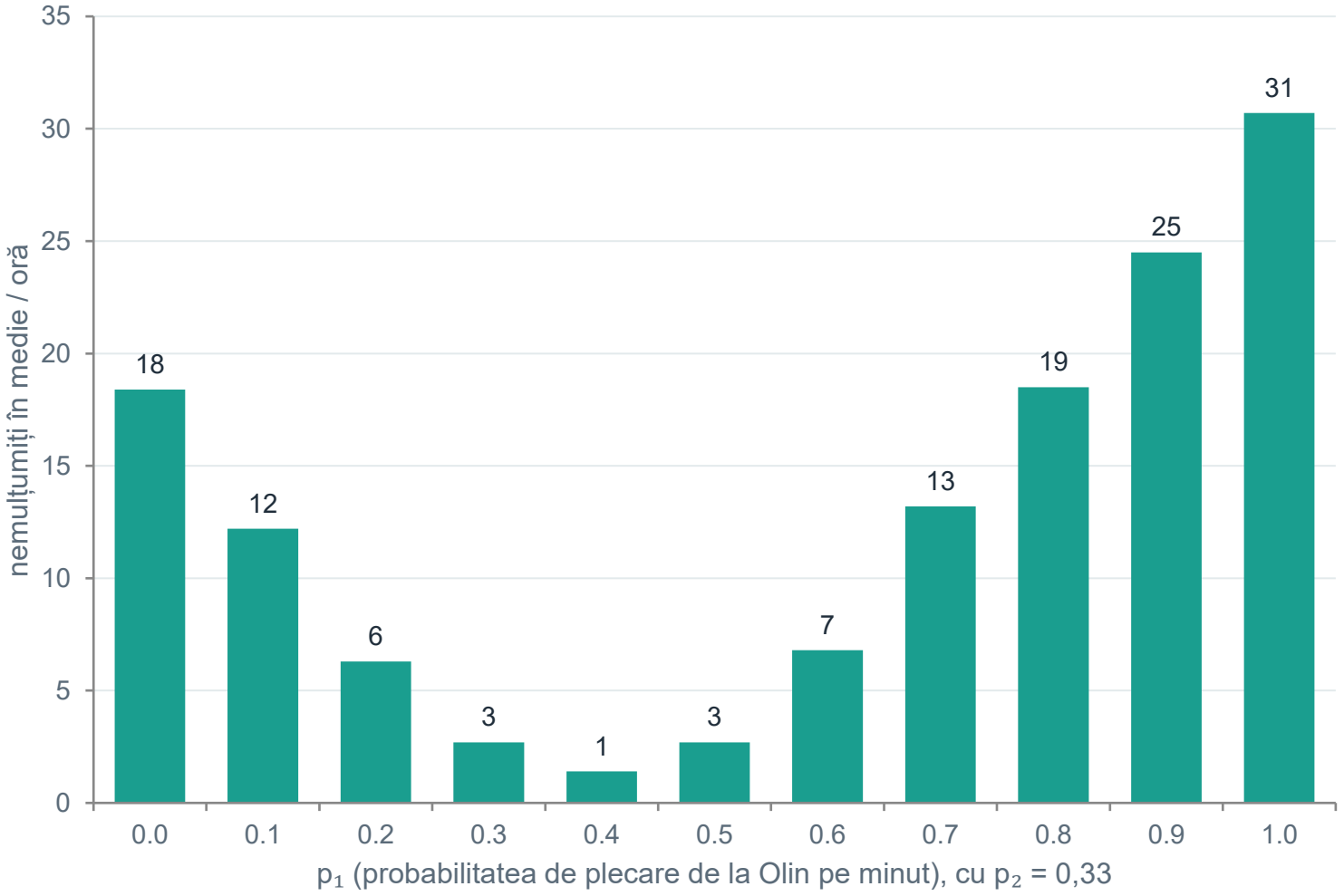
Experiment: cum depind nemulțumiții de p_1?

```
import numpy as np

valori_p1 = np.linspace(0, 1, 11)
medii = []
for p1 in valori_p1:
    # 200 de replicări pentru fiecare valoare
    rezultate = [simuleaza(p1, 0.33)[1]
                 for _ in range(200)]
    medii.append(np.mean(rezultate))
```

- Variem un singur parametru: **p_1** între 0 și 1
- Pentru fiecare valoare: **200 de replicări**
- Păstrăm media numărului de nemulțumiți
- Aceasta este o **analiză de sensibilitate** simplă

Rezultat: nemulțumiți în funcție de p_1



- Curbă în formă de **U**
- Minim când fluxurile sunt **echilibrate** ($p_1 \approx p_2$)
- p_1 mic: Wellesley rămâne fără biciclete
- p_1 mare: Olin rămâne fără biciclete
- Idee de politică: **rebalansare** periodică

Verificare

? În modelul bike share, cu $\mathbf{p_1 = 1}$, $\mathbf{p_2 = 0}$, 10 biciclete inițial la Olin și **60 de minute**, câți clienți nemulțumiți ar trebui să raporteze programul?

A 0

B 10

C 50

D 60

Verificarea modelului bike share

Test	Ce ne așteptăm să vedem
Conservare	olin + wellesley = 12 la fiecare pas
Fără valori negative	nicio stație nu are sub 0 biciclete
Caz extrem: $p_1 = p_2 = 0$	starea nu se schimbă deloc; 0 nemulțumiți
Caz extrem: $p_1 = 1$, $p_2 = 0$	Olin se golește după 10 minute; apoi câte un nemulțumit pe minut (50 în total)
Reproductibilitate	același seed → exact aceeași rulare

Validarea modelului bike share

- Comparăm cu **date reale** de utilizare: rețelele de bike sharing publică date de călătorii
- Estimăm p_1 și p_2 **din date**, pe intervale orare (la laborator: date din Londra)
- Verificăm dacă modelul reproduce **orele** în care stațiile se golesc
- Dacă nu: revizuim **ipotezele** – de ex. p_1 nu este constant pe parcursul zilei

L **Legătura cu laboratorul**

La laborator veți extinde modelul: probabilități diferite pe intervale orare, mentenanță, rebalansare.

Ce am învățat din exemplu

1 Întâi întrebarea

Măsura de performanță (nemulțumiții) a decis ce înregistrăm.

2 Model mic, util

Sub 30 de linii de cod, dar deja arată o idee de politică: rebalansarea.

3 Aleatorul cere statistică

O rulare e o anecdotă; 200 de replicări dau o concluzie.

4 Verificare simplă

Conservarea și cazurile extreme prind majoritatea bug-urilor.

Cum arată un raport de simulare bun

1 Întrebarea

O frază: ce decizie sprijină studiul și ce măsură de performanță folosim.

2 Ipotezele

Lista a ceea ce am inclus, ce am ignorat și de unde vin datele.

3 Rezultatele

Medii **cu intervale de încredere**, grafice simple, comparații între variante.

4 Limitele

Unde nu mai este valabil modelul și ce ar putea schimba concluzia.

Verificare

?

Simularea unei bănci nu are niciun bug, dar prezice cozi de 3 ori mai mici decât cele reale, pentru că presupune că toți clienții fac operații scurte. Problema este de:

A verificare

B validare

C implementare

D generare a numerelor aleatoare

Frontiera

Unde se folosește simularea azi

1. Prognoza meteo cu inteligență artificială
2. Digital twins
3. Roboți antrenați în simulator
4. Agenți generativi cu modele de limbaj

Prognoza meteo cu AI: GenCast (2024)

97,2%

din 1 320 de ținte de prognoză: GenCast mai precis decât sistemul de referință ENS (ECMWF)

8 min

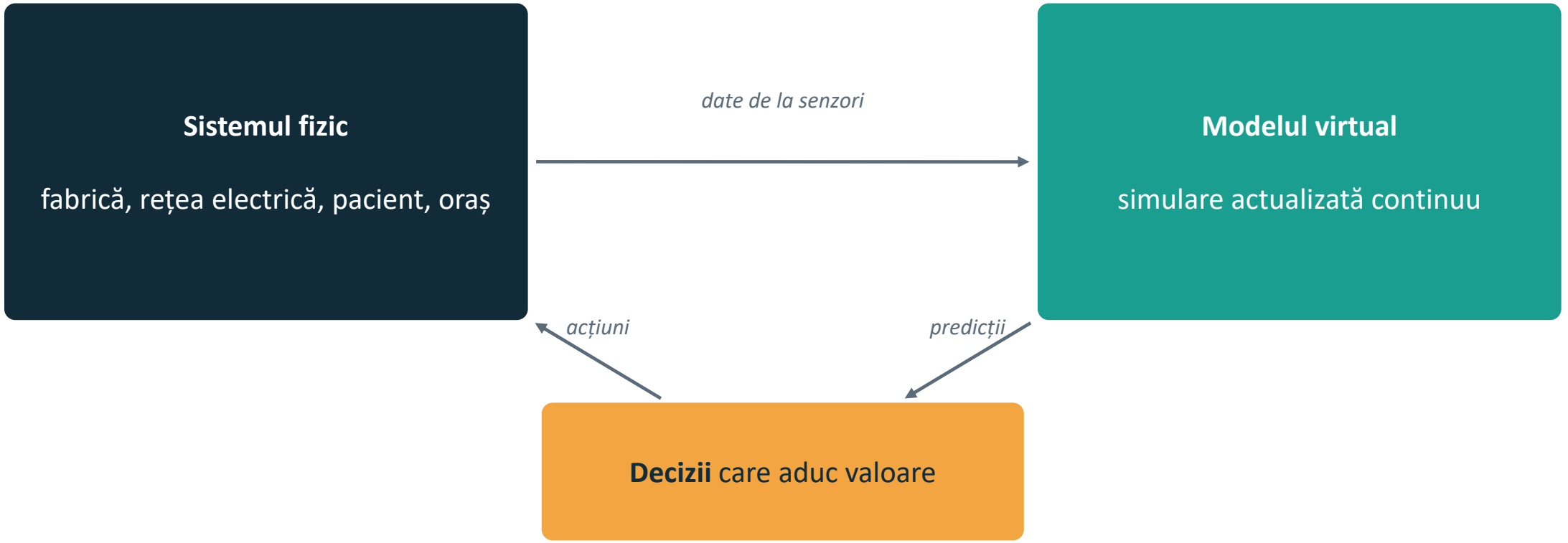
pentru o prognoză pe 15 zile, pe un singur cip TPU

40 ani

de date „reanalizate” – produse la rândul lor cu simulări numerice – folosite la antrenare

- Modelul AI **învață** din decenii de simulări și observații, apoi le **înlocuiește** la rulare
- Sursa: Price et al., Nature, 2024; Google DeepMind

Digital twin: modelul care rămâne conectat



Definiție: model virtual actualizat dinamic cu date din sistemul fizic, cu capacitate predictivă, care informează decizii (National Academies, 2023)

Roboți antrenați în simulator

Ideea

- robotul învață prin **încercare și eroare** (învățare prin întărire)
- în lumea reală: lent, scump, periculos
- în simulator: **mii de copii** în paralel, pe GPU
- simulatoare: MuJoCo, NVIDIA Isaac Lab

Provocarea: sim-to-real

- simulatorul nu e perfect: frecări, senzori, întârzieri
- politica învățată poate eșua în realitate
- soluție: **randomizarea domeniului** – variem parametrii simulării
- aceeași idee: date sintetice pentru mașini autonome

Agenți generativi cu modele de limbaj

Ce s-a făcut

- Park et al. (2023): **25 de agenți** cu LLM într-un orășel virtual
- agenții își planifică ziua, discută, își amintesc
- au organizat „spontan” o petrecere
- folosiți azi pentru simulări sociale și de piață

Problema deschisă

- comportamentul pare **realist** – dar este corect?
- modelele de limbaj au **biasuri** și sunt greu de explicat
- rezultatele sunt greu de **validat** cu date reale
- lecția de azi rămâne valabilă: fără validare, doar o poveste frumoasă

Simularea în industrie

1 Fabrici și logistică

Linii de producție, depozite, lanțuri de aprovizionare.

2 Sănătate

Secții de urgențe, programări, răspândirea bolilor.

3 Energie

Rețele electrice cu surse regenerabile și baterii.

4 Transport

Trafic urban, aeroporturi, orare de tren.

5 IT și cloud

Capacitatea centrelor de date, rețele, testarea sistemelor distribuite.

6 Jocuri și film

Motoare fizice, fluide, mulțimi de personaje.

Uneltele pe care le vom întâlni

Unealtă	La ce folosește	În curs / laborator
Python + NumPy / SciPy	calcul numeric, distribuții, ecuații diferențiale	toate săptămânile
Matplotlib	grafice pentru rezultate	toate săptămânile
SimPy	simulare cu evenimente discrete, orientată pe procese	S5, laborator
Mesa	modele cu agenți în Python	S9
Pygame	vizualizare interactivă (ex. Boids)	laborator
AnyLogic, NetLogo	medii specializate – prezentate, nu obligatorii	S5, S9

De ce Python în acest curs?

- Același limbaj pentru **model, analiză statistică și grafice**
- Biblioteci mature și gratuite: NumPy, SciPy, SimPy, Mesa
- Codul este aproape **pseudocod** – ușor de citit pe un slide
- Folosit și în industrie, și în cercetare, inclusiv în AI
- Limitare: bucle lente – soluții: **vectorizare** cu NumPy, biblioteci compilate (săptămâna 8)

Important

Conceptele din curs nu depind de limbaj. Un simulator cu evenimente discrete arată la fel în Python, Java sau C++.

Verificare

? Care este diferența esențială dintre un **digital twin** și o simulare obișnuită?

A Digital twin-ul folosește grafică 3D

B Digital twin-ul este actualizat continuu cu date din sistemul real și informează decizii

C Digital twin-ul nu conține elemente aleatoare

D Digital twin-ul rulează doar pe supercalculatoare

Rezumatul săptămânii 1

- ✓ Simulăm când experimentul real este imposibil, scump, lent sau periculos
- ✓ **Sistem** – **model** – **simulare**; starea, evenimentele și activitățile
- ✓ Clasificăm modelele: static/dinamic, determinist/stocastic, timp și stare discrete/continue
- ✓ Un studiu de simulare are **10 etape**; cele mai neglijate: formularea problemei și validarea
- ✓ **Verificare** = am construit corect modelul? **Validare** = am construit modelul corect?
- ✓ Modelele stocastice cer **multe rulări** și statistică (vezi bike share)
- ✓ Frontiera: AI + simulare, digital twins, roboți, agenți generativi

Săptămâna viitoare

Aleatorul în simulare și metoda Monte Carlo

- Distribuțiile de care avem nevoie în simulare
- Cum produce un calculator numere „aleatoare”
- Estimări prin eșantionare, cu interval de încredere

Cum poți estima numărul π aruncând puncte la întâmplare?

Bibliografie pentru săptămâna 1

- A. M. Law – Simulation Modeling and Analysis, ed. 6, McGraw Hill, 2024 – cap. 1
- J. Banks, J. S. Carson, B. L. Nelson, D. M. Nicol – Discrete-Event System Simulation, ed. 5, Pearson, 2010 – cap. 1
- A. B. Downey – Modeling and Simulation in Python, No Starch Press, 2023 – cap. 1–3 (gratuită online: allendowney.github.io/ModSimPy)
- Ș. Holban – Tehnici de modelare și simulare, cap. 1 (materiale de curs)
- National Academies – Foundational Research Gaps and Future Directions for Digital Twins, 2023
- I. Price et al. – Probabilistic weather forecasting with machine learning (GenCast), Nature, 2024
- J. S. Park et al. – Generative Agents: Interactive Simulacra of Human Behavior, UIST 2023
- Prezentare realizata cu ajutorul Claude Fable 5.1 și Claude Opus 5.5